# Predicting Bank Term Deposit Subscription
# Case Study Report

R. Promkam

March 4, 2026

**Abstract**

We analyse a bank marketing dataset containing 11,162 records with 16 demographic, financial, and campaign-related features. The task is a **classification** problem: predicting whether a client will subscribe to a term deposit. The dataset contains no missing values and no duplicate rows. After one-hot encoding 9 categorical features, we obtain 42 input features. We train and tune seven classification models via grid-search cross-validation using the weighted $F_1$ score. The best-performing model is a gradient boosting classifier, achieving a test $F_1$ of 0.8662 and a test ROC-AUC of 0.9302. The results confirm that call duration and previous campaign outcome are the strongest predictors of deposit subscription.

## Contents

**1 Introduction** **2**

**2 Data Cleaning** **2**
2.1 Column information . . . 4
2.2 Descriptive statistics . . . 5
2.3 Missing values . . . 5
2.4 Duplicate check . . . 6

**3 Exploratory Data Analysis** **6**
3.1 Target distribution . . . 6
3.2 Numerical feature distributions . . . 7
3.3 Correlation heatmap . . . 11
3.4 Categorical features vs target . . . 12
3.5 Top-2 correlated numerical features . . . 16

**4 Feature Engineering and Data Splitting** **17**
4.1 One-hot encoding and target encoding . . . 17
4.2 Stratified train/test split . . . 17
4.3 $F_\beta$ score . . . 18
4.4 Feature scaling . . . 18

**5 Model Training** **18**
5.1 Logistic regression . . . 19
5.2 K-nearest neighbours . . . 19
5.3 Support vector classifier . . . 20
5.4 Decision tree . . . 20
5.5 Random forest . . . 21
5.6 Gradient boosting . . . 21
5.7 Gaussian naive Bayes . . . 22

**6 Model Evaluation** **22**
6.1 Per-model test results . . . 23
6.2 Summary of test results . . . 23

**7 Model Comparison** **24**
7.1 CV $F_1$ vs test $F_1$ . . . 24
7.2 Test ROC-AUC comparison . . . 25
7.3 Confusion matrix . . . 26
7.4 ROC curve . . . 27
7.5 Best model selection . . . 28

**8 Model Deployment** **28**
8.1 Saving the artifact . . . 28
8.2 Loading and inspecting the artifact . . . 29
8.3 Prediction on a new observation . . . 30

**9 Conclusion** **31**

# 1 Introduction

Bank marketing campaigns [4] aim to persuade clients to subscribe to financial products such as term deposits. Identifying which clients are most likely to respond positively allows banks to allocate marketing resources efficiently, reducing costs while maximising conversion rates.

The shift from rule-based targeting to machine learning models enables data-driven decision-making that captures complex, non-linear relationships among client demographics, account information, and campaign characteristics. Accurate prediction of subscription likelihood informs more personalised outreach and improves overall campaign effectiveness.

In this case study we formulate the problem as a supervised classification task: given a feature vector describing a client and the associated marketing campaign, we predict whether the client will subscribe to a term deposit. We pursue the following objectives:

(i) Clean the raw dataset and perform exploratory data analysis to identify key predictors.

(ii) Engineer features via one-hot encoding and prepare a stratified train/test split.

(iii) Train seven classification models with hyperparameter tuning via five-fold cross-validation.

(iv) Evaluate all models on a held-out test set and select the best performer.

(v) Deploy the chosen model as a serialised artifact for future inference.

The remainder of this report is organised as follows. Section 2 describes data cleaning. Section 3 presents the exploratory data analysis. Section 4 covers feature engineering and the train/test split. Sections 5 and 6 detail model training and evaluation, respectively. Section 7 compares all models. Section 8 demonstrates model deployment. Section 9 concludes with practical implications and future directions.

# 2 Data Cleaning

We begin by importing the required libraries and loading the dataset. The imported libraries serve specific roles in our pipeline:

- `numpy` [1] and `pandas` [3] provide core data structures for manipulation and numerical operations.

- `matplotlib` [2] and `seaborn` [8] are used for visualising data distributions and relationships.
- `sklearn` (Scikit-learn) [6] supplies the machine learning algorithms, preprocessing utilities like `StandardScaler`, evaluation metrics such as `accuracy_score`, `fbeta_score`, and `roc_auc_score`, and model selection tools like `GridSearchCV`.
- `os`, `warnings`, and `pickle` [7] handle file system operations, suppress routine warnings, and allow us to serialise our final model to disk, respectively.

```python
import os
import warnings
import pickle
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import (
    accuracy_score, fbeta_score, make_scorer,
    confusion_matrix, ConfusionMatrixDisplay,
    roc_curve, auc, roc_auc_score
)

np.random.seed(42)
plt.rcParams.update({'figure.dpi': 120, 'font.size': 11})
os.makedirs('figures', exist_ok=True)
warnings.filterwarnings('ignore')
```

We load the dataset [5] from a CSV file into a pandas DataFrame and inspect its shape and first few rows to confirm that the data has been read correctly.

```python
df = pd.read_csv('bank.csv')
print(f"Shape: {df.shape}")
df.head()
```

The DataFrame `df` has shape $(11162, 17)$, comprising 11,162 observations and 17 columns. The first five rows are shown below.

| age | job | marital | education | balance | housing | loan | deposit |
|---|---|---|---|---|---|---|---|
| 59 | admin. | married | secondary | 2,343 | yes | no | yes |
| 56 | admin. | married | secondary | 45 | no | no | yes |
| 41 | technician | married | secondary | 1,270 | yes | no | yes |
| 55 | services | married | secondary | 2,476 | yes | no | yes |
| 54 | admin. | married | tertiary | 184 | no | no | yes |

The 17 columns are as follows:

- `age` (int64) — client age in years.
- `job` (object) — type of job (e.g., admin., technician, management).
- `marital` (object) — marital status (married, single, divorced).
- `education` (object) — education level (primary, secondary, tertiary, unknown).
- `default` (object) — whether the client has credit in default (yes/no).
- `balance` (int64) — average yearly account balance in euros.
- `housing` (object) — whether the client has a housing loan (yes/no).
- `loan` (object) — whether the client has a personal loan (yes/no).
- `contact` (object) — contact communication type (cellular, telephone, unknown).
- `day` (int64) — last contact day of the month.
- `month` (object) — last contact month of the year.
- `duration` (int64) — last contact duration in seconds.
- `campaign` (int64) — number of contacts during this campaign.
- `pdays` (int64) — number of days since the client was last contacted from a previous campaign ($-1$ means not previously contacted).
- `previous` (int64) — number of contacts before this campaign.
- `poutcome` (object) — outcome of the previous marketing campaign.
- `deposit` (object) — whether the client subscribed to a term deposit (yes/no); the target variable.

## 2.1 Column information

We inspect the column data types and non-null counts to verify structural integrity before proceeding with any transformations.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11162 entries, 0 to 11161
Data columns (total 17 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   age         11162 non-null  int64
 1   job         11162 non-null  object
 2   marital     11162 non-null  object
 3   education   11162 non-null  object
 4   default     11162 non-null  object
 5   balance     11162 non-null  int64
 6   housing     11162 non-null  object
 7   loan        11162 non-null  object
 8   contact     11162 non-null  object
 9   day         11162 non-null  int64
 10  month       11162 non-null  object
 11  duration    11162 non-null  int64
```

```
 12  campaign   11162 non-null  int64
 13  pdays      11162 non-null  int64
 14  previous   11162 non-null  int64
 15  poutcome   11162 non-null  object
 16  deposit    11162 non-null  object
dtypes: int64(7), object(10)
memory usage: 1.4+ MB
```

All 11,162 entries are non-null across every column. The dataset contains seven integer columns and ten categorical object columns.

## 2.2 Descriptive statistics

We compute summary statistics for all numerical columns to understand the central tendency, spread, and range of each feature.

```python
df.describe()
```

| | age | balance | day | duration | campaign | pdays | previous |
|---|---|---|---|---|---|---|---|
| count | 11,162 | 11,162 | 11,162 | 11,162 | 11,162 | 11,162 | 11,162 |
| mean | 41.23 | 1,528.54 | 15.66 | 371.99 | 2.51 | 51.33 | 0.83 |
| std | 11.91 | 3,225.41 | 8.42 | 347.13 | 2.72 | 108.76 | 2.29 |
| min | 18 | $-6,847$ | 1 | 2 | 1 | $-1$ | 0 |
| 25% | 32 | 122 | 8 | 138 | 1 | $-1$ | 0 |
| 50% | 39 | 550 | 15 | 255 | 2 | $-1$ | 0 |
| 75% | 49 | 1,708 | 22 | 496 | 3 | 20.75 | 1 |
| max | 95 | 81,204 | 31 | 3,881 | 63 | 854 | 58 |

The mean age is approximately 41 years, ranging from 18 to 95. The account balance has a large standard deviation (3,225.41) relative to its mean (1,528.54), indicating substantial variability. The `duration` column records the last contact duration in seconds, with a mean of 372 seconds (approximately 6 minutes) and a maximum of 3,881 seconds. The `pdays` column has a minimum of $-1$, which indicates clients who were not previously contacted.

## 2.3 Missing values

We check for missing data across all columns to determine whether imputation is necessary before modelling.

```python
print("Missing values per column:")
print(df.isnull().sum())
print(f"\nTotal missing: {df.isnull().sum().sum()}")
```

| Column | Missing |
|---|---|
| age | 0 |
| job | 0 |
| marital | 0 |
| education | 0 |
| default | 0 |
| balance | 0 |
| housing | 0 |
| loan | 0 |
| contact | 0 |
| day | 0 |
| month | 0 |
| duration | 0 |
| campaign | 0 |
| pdays | 0 |
| previous | 0 |
| poutcome | 0 |
| deposit | 0 |
| Total | 0 |

No missing values are present in any column. We proceed directly to duplicate detection.

### 2.4 Duplicate check

We detect exact duplicate rows to determine whether deduplication is necessary.

```
n_dup = df.duplicated().sum()
print(f"Shape before cleaning: {df.shape}")
print(f"Duplicate rows found:  {n_dup}")

if n_dup > 0:
    print("\nSample duplicates:")
    display(df[df.duplicated(keep='first')].head())

df = df.drop_duplicates()
print(f"\nShape after cleaning:  {df.shape}")
```

```
Shape before cleaning: (11162, 17)
Duplicate rows found:  0

Shape after cleaning:  (11162, 17)
```

No duplicate rows are found. The dataset retains all 11,162 records for subsequent analysis.

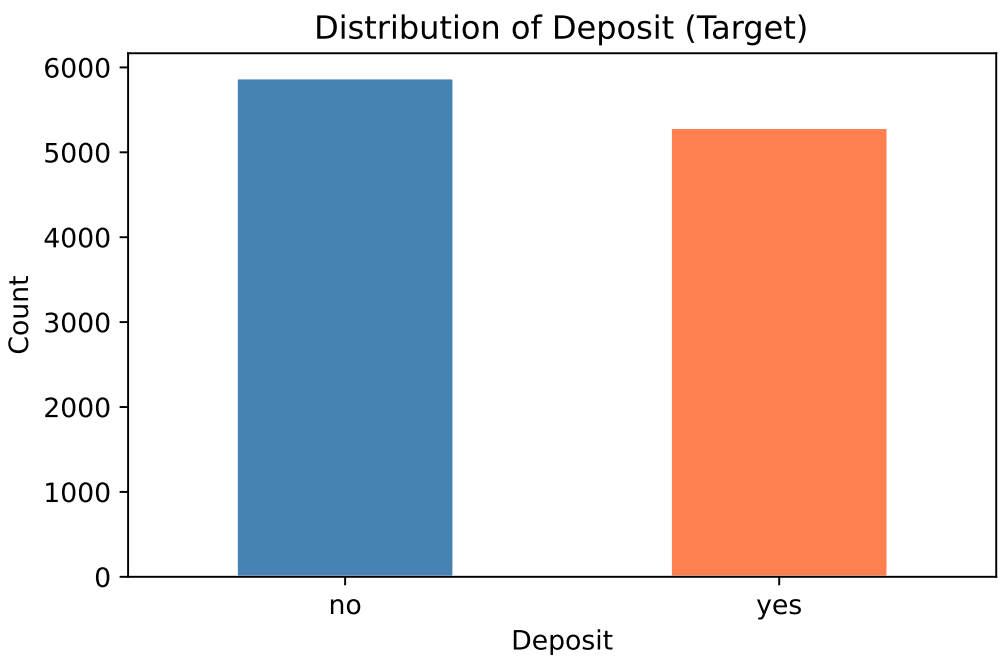
## 3 Exploratory Data Analysis

We examine the target distribution, individual feature distributions, correlations among numerical variables, and the relationship between categorical features and deposit subscription.

### 3.1 Target distribution

We begin by plotting the bar chart of the target variable to assess the class balance.

```python
fig, ax = plt.subplots(figsize=(6, 4))
counts = df['deposit'].value_counts()
counts.plot(kind='bar', ax=ax, color=['steelblue', 'coral'], edgecolor='white')
ax.set_xlabel('Deposit')
ax.set_ylabel('Count')
ax.set_title('Distribution of Deposit (Target)')
ax.set_xticklabels(ax.get_xticklabels(), rotation=0)
plt.tight_layout()
fig.savefig('figures/fig_01_target_distribution.pdf', bbox_inches='tight')
plt.show()

print(f"Class counts:\n{counts}")
ratio = counts.min() / counts.max()
print(f"\nClass balance ratio (minority / majority): {ratio:.2f}")
```



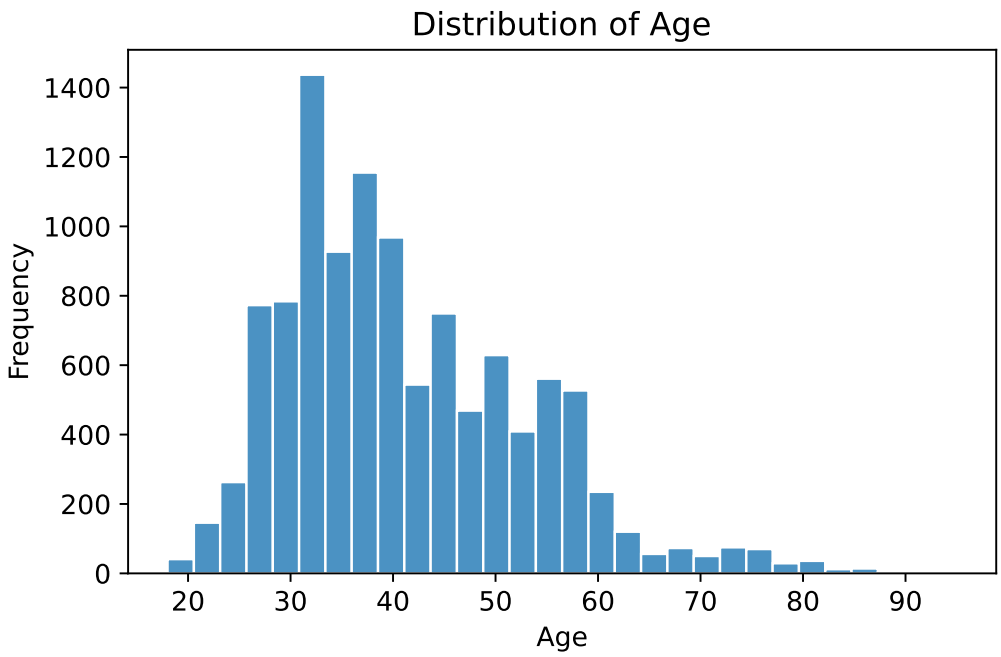

The target variable has two classes: "no" ($n = 5{,}873$) and "yes" ($n = 5{,}289$). The class balance ratio is 0.90, indicating a roughly balanced dataset. No resampling or class-weight adjustments are necessary.

### 3.2 Numerical feature distributions

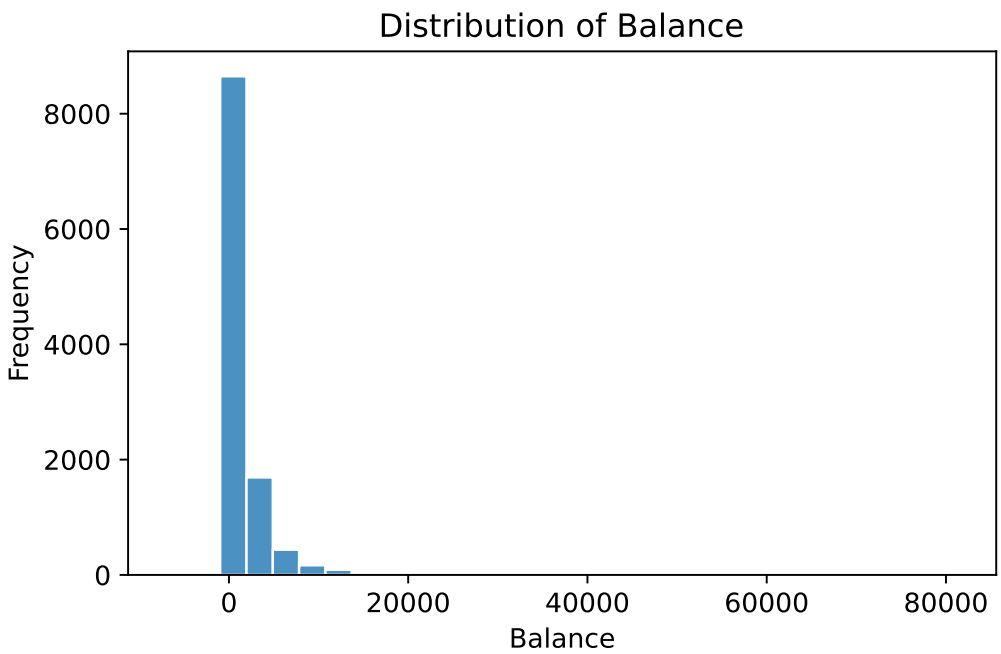
We visualise the distributions of the seven numerical features to understand their spread and identify potential outliers.

```python
numerical_cols = ['age', 'balance', 'day', 'duration',
                  'campaign', 'pdays', 'previous']
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g']

for col, label in zip(numerical_cols, labels):
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.hist(df[col], bins=30, edgecolor='white', alpha=0.8)
    ax.set_xlabel(col.capitalize())
    ax.set_ylabel('Frequency')
    ax.set_title(f'Distribution of {col.capitalize()}')
    plt.tight_layout()
```

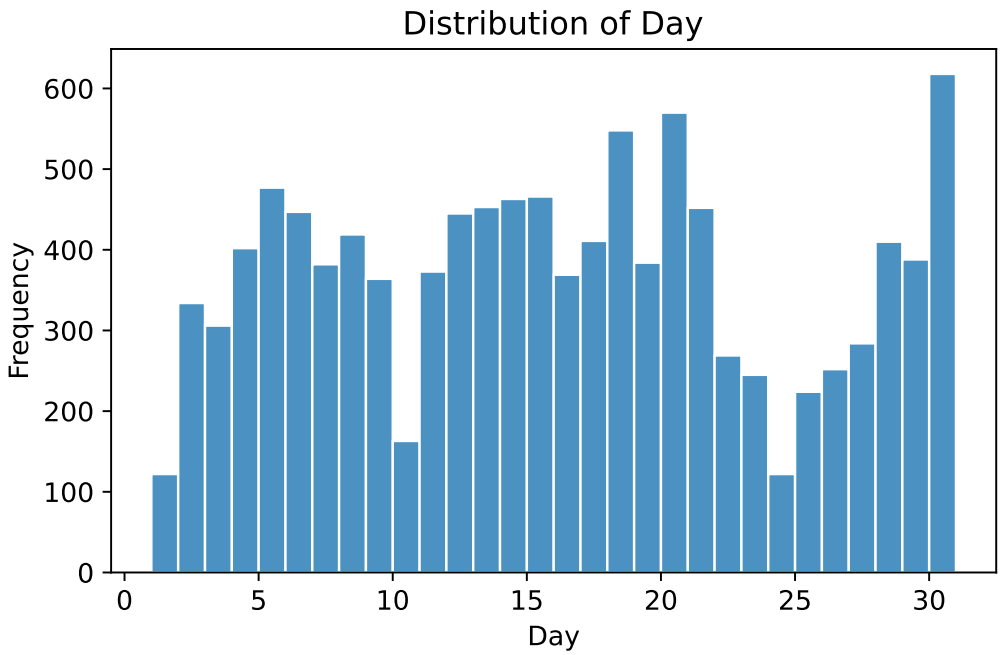
```
    fig.savefig(f'figures/fig_02{label}_distribution_{col}.pdf',
                bbox_inches='tight')
    plt.show()
```



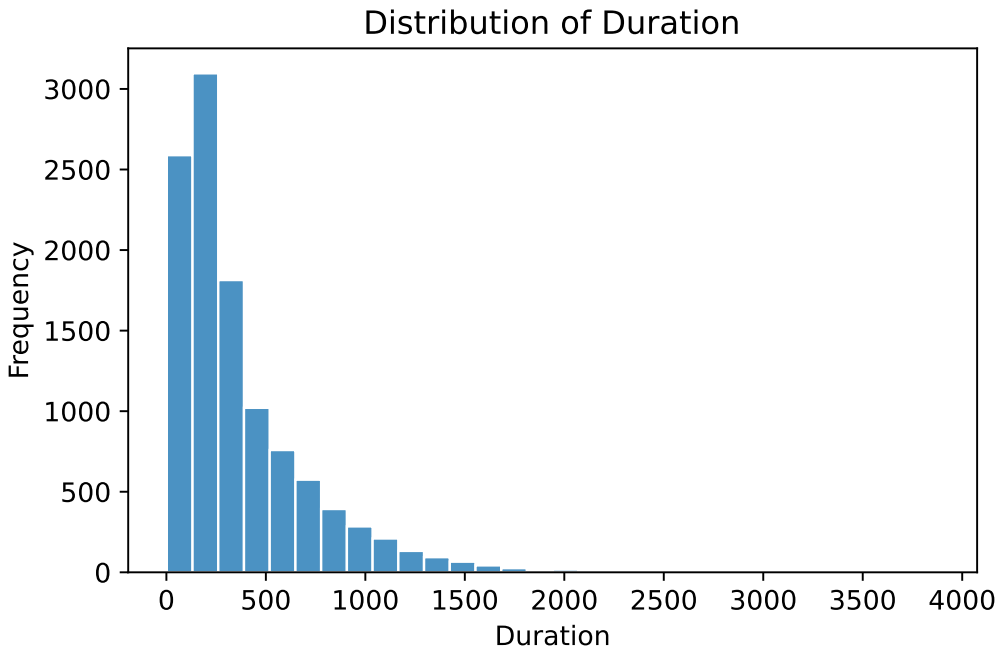

The age distribution spans 18 to 95 years with a mean of approximately 41 years. The distribution is slightly right-skewed with a concentration in the 30–50 range.



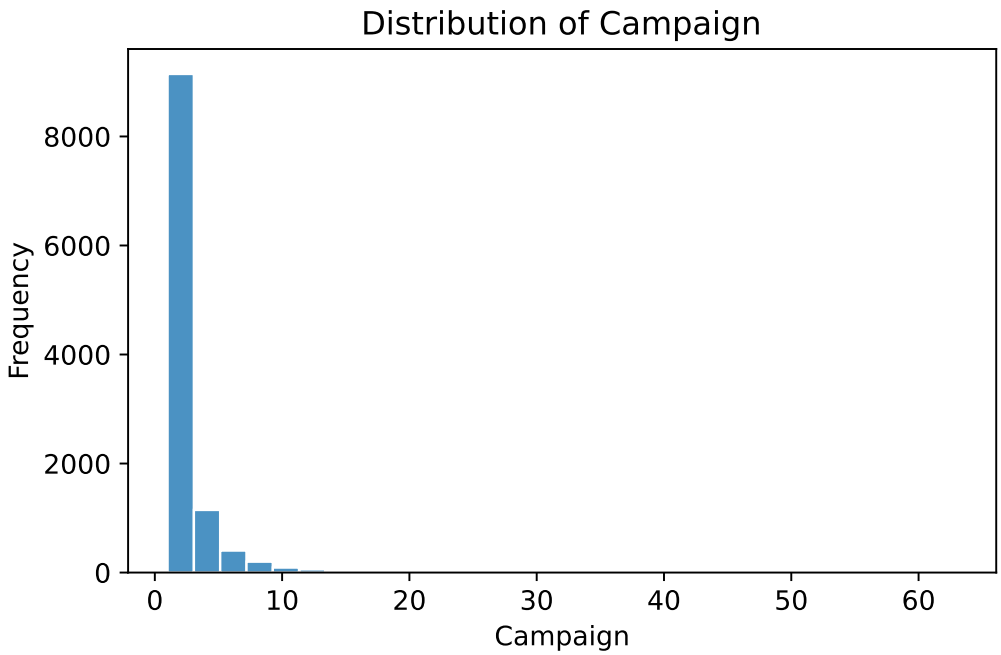

The balance distribution is heavily right-skewed, with most clients having a balance below 5,000 euros. A few extreme values reach over 80,000 euros, and negative balances are present.



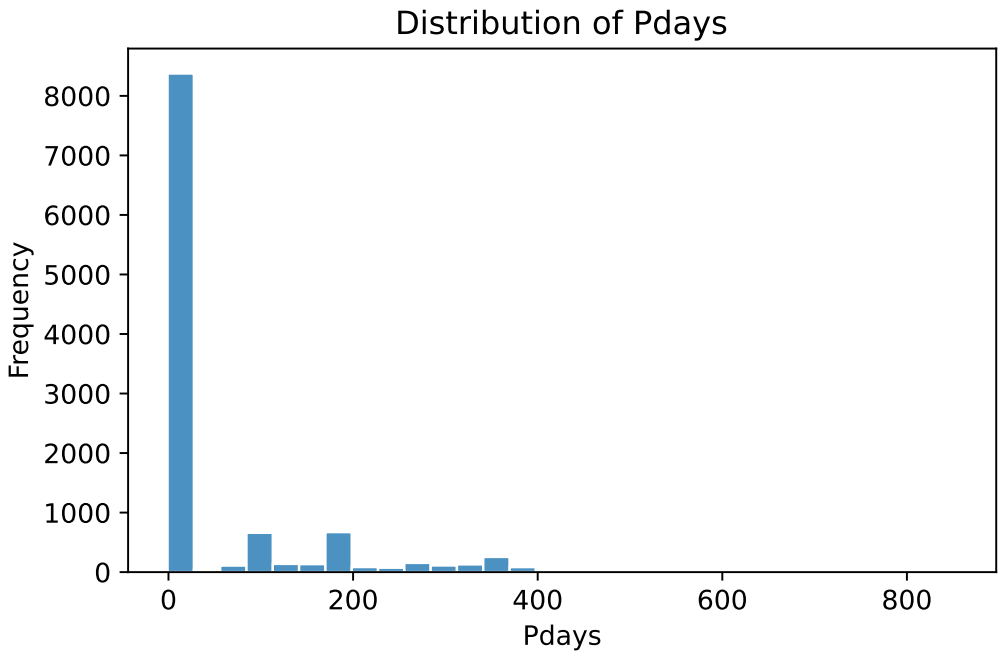

The day-of-month distribution is roughly uniform across 1–31, as expected for calendar days.



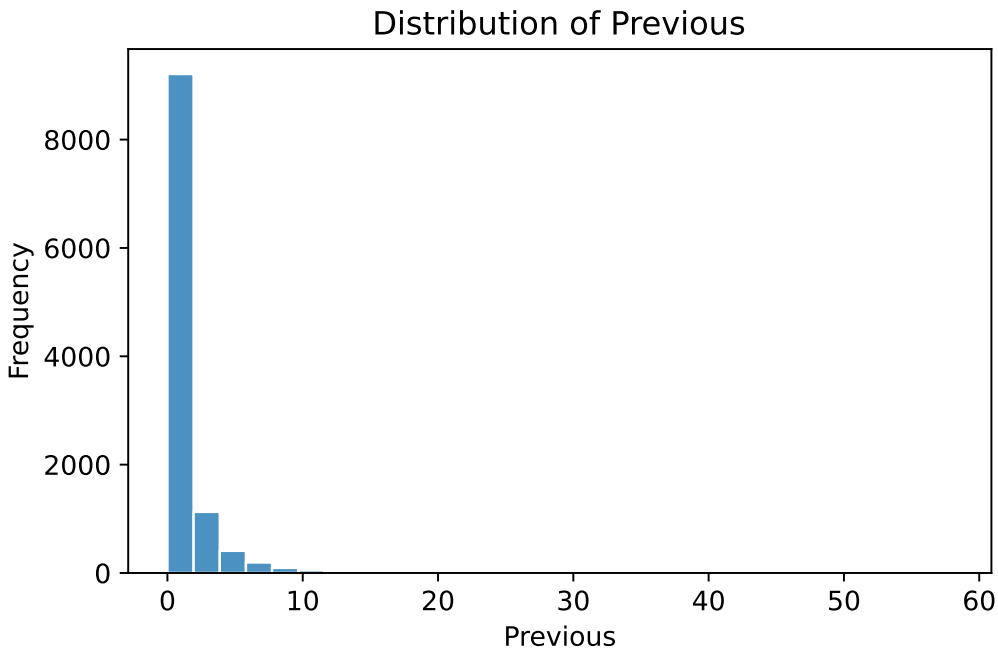

Call duration is right-skewed with a median of 255 seconds. Most calls last under 500 seconds, but some extend beyond 2,000 seconds.




The number of contacts during the current campaign is highly right-skewed, with the vast majority of clients contacted only 1–3 times.




The `pdays` distribution is dominated by the value $-1$ (not previously contacted), with a long tail for clients who were contacted in prior campaigns.

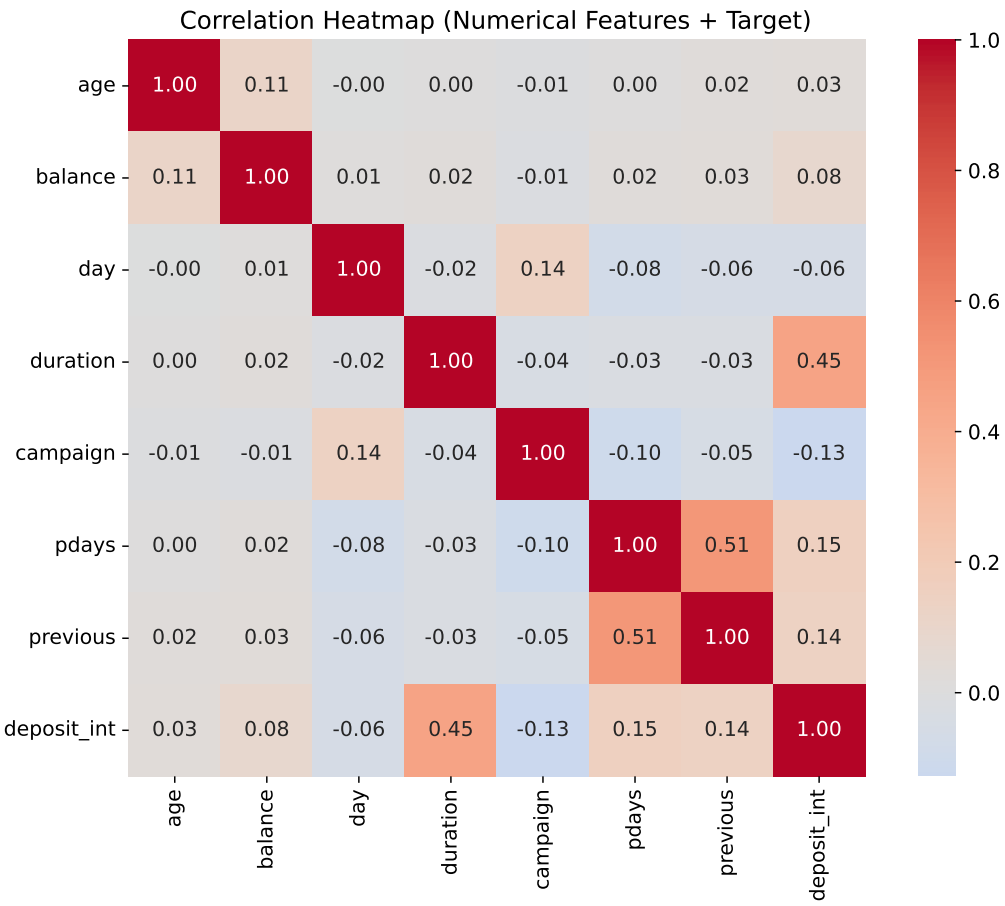



The number of previous contacts is heavily concentrated at zero, with very few clients having more than 5 prior contacts.

### 3.3 Correlation heatmap

We compute the pairwise Pearson correlation coefficients among the numerical features and the binary-encoded target variable to identify linear relationships.

```
df_corr = df.copy()
df_corr['deposit_int'] = (df_corr['deposit'] == 'yes').astype(int)

corr_cols = numerical_cols + ['deposit_int']
corr = df_corr[corr_cols].corr()

fig, ax = plt.subplots(figsize=(9, 7))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0,
            square=True, ax=ax)
ax.set_title('Correlation Heatmap (Numerical Features + Target)')
plt.tight_layout()
fig.savefig('figures/fig_03_correlation_heatmap.pdf', bbox_inches='tight')
plt.show()

target_corr = corr['deposit_int'].drop('deposit_int').abs().sort_values(
    ascending=False)
print("Correlation with deposit (absolute):")
print(target_corr)
top2 = target_corr.index[:2].tolist()
print(f"\nTop-2 correlated features: {top2}")
```

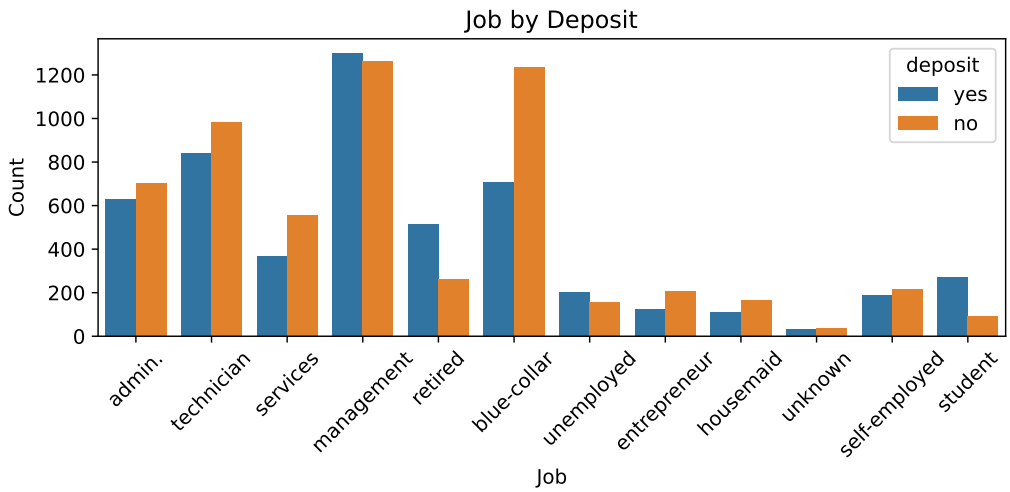



The heatmap reveals that `duration` has the strongest linear correlation with the target ($|r| = 0.45$), followed by `pdays` (0.15), `previous` (0.14), and `campaign` (0.13). The remaining numerical features—`balance` (0.08), `day` (0.06), and `age` (0.03)—show weak linear association with deposit subscription.

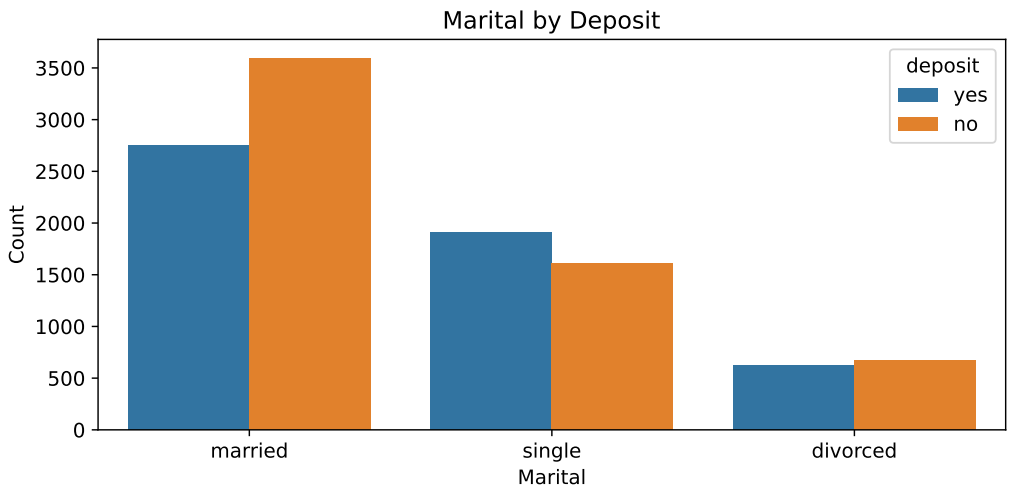
### 3.4 Categorical features vs target

We use count plots to examine how the categorical features relate to the target variable.

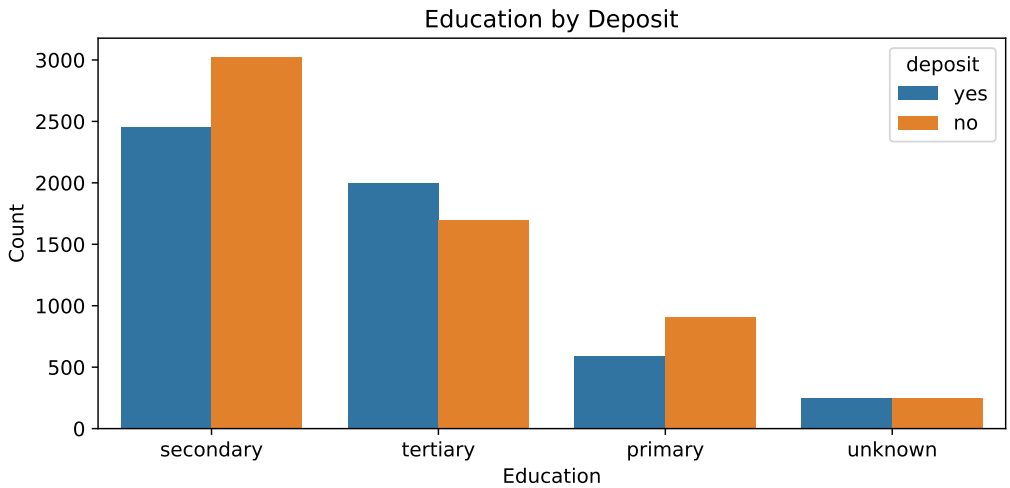
```python
categorical_cols = ['job', 'marital', 'education', 'default', 'housing',
                    'loan', 'contact', 'month', 'poutcome']
cat_labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i']

for col, label in zip(categorical_cols, cat_labels):
    fig, ax = plt.subplots(figsize=(8, 4))
    sns.countplot(data=df, x=col, hue='deposit', ax=ax)
    ax.set_title(f'{col.capitalize()} by Deposit')
    ax.set_xlabel(col.capitalize())
    ax.set_ylabel('Count')
    if df[col].nunique() > 5:
        ax.tick_params(axis='x', rotation=45)
    plt.tight_layout()
    fig.savefig(f'figures/fig_04{label}_countplot_{col}.pdf',
                bbox_inches='tight')
    plt.show()
```



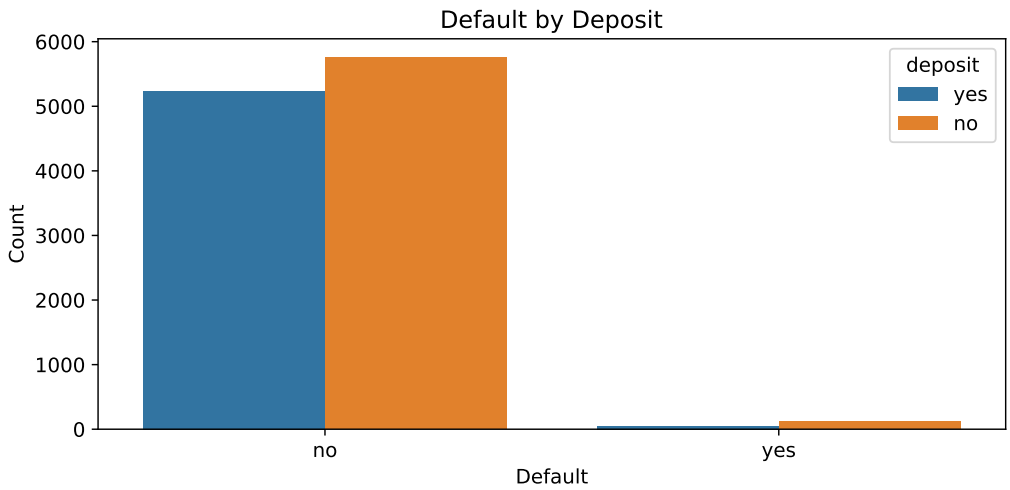

Job type shows notable variation in deposit rates. Students and retired clients have higher subscription rates, while blue-collar workers subscribe less frequently.



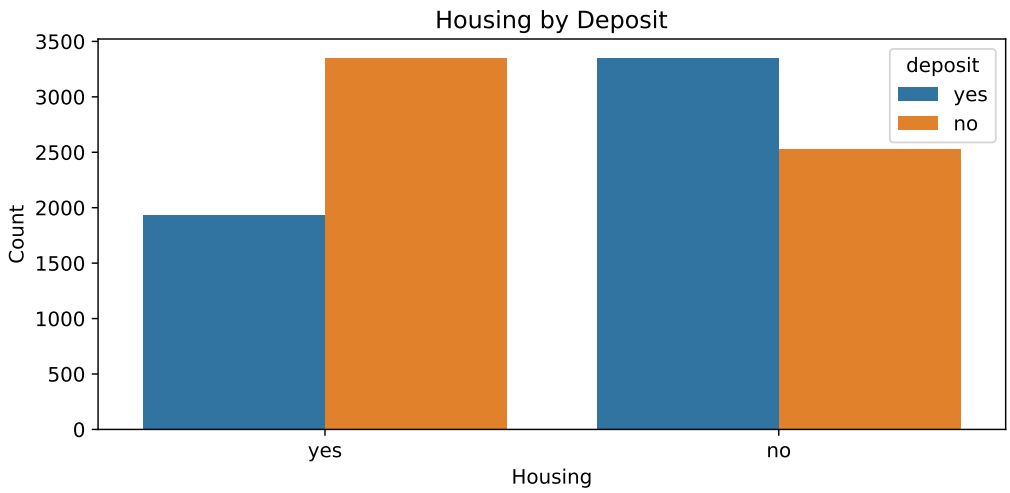

Single clients show a slightly higher deposit subscription rate compared to married and divorced clients.



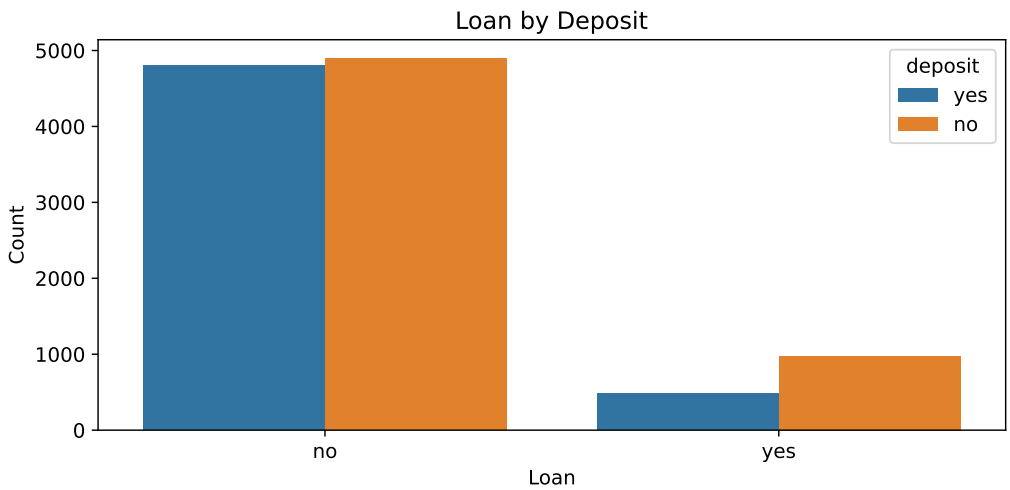

Clients with tertiary education subscribe at a higher rate than those with primary or secondary education.



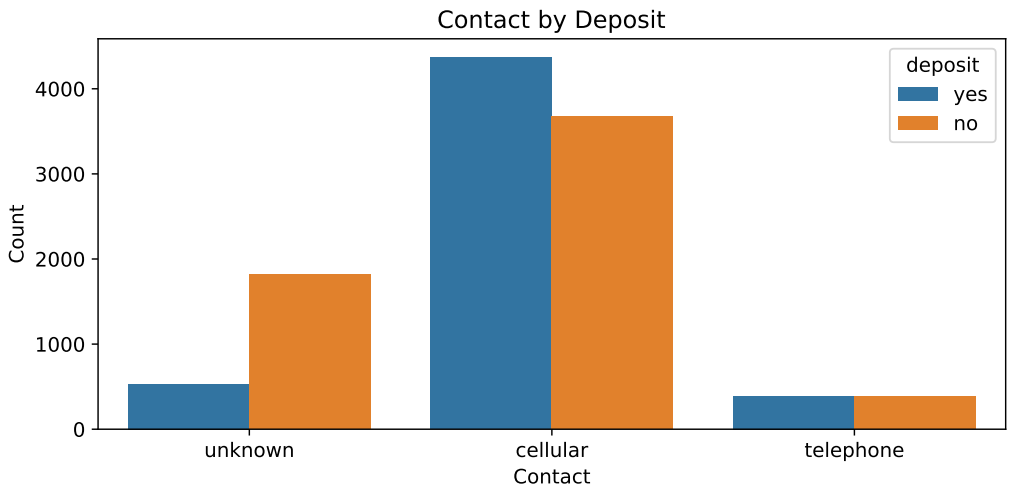

Very few clients have credit in default, and those who do show slightly lower subscription rates.



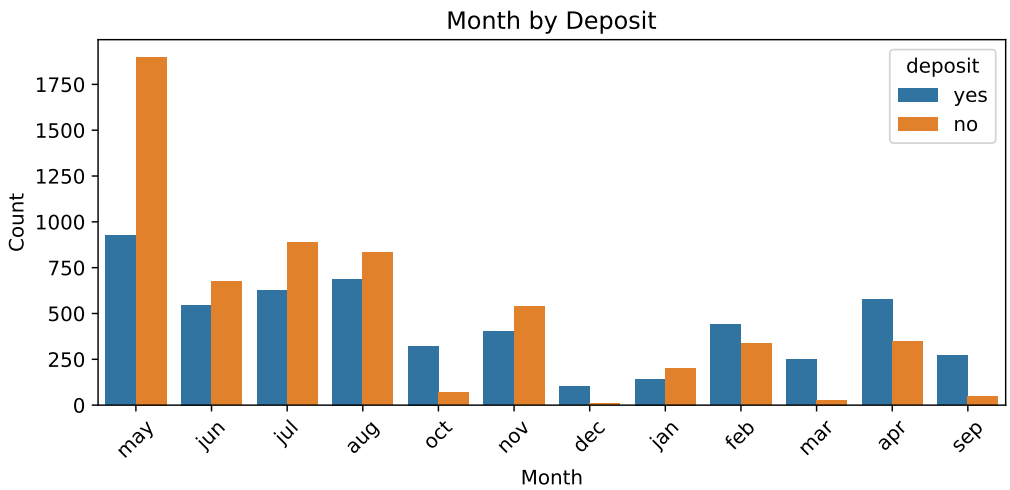

Clients without a housing loan subscribe at a higher rate than those with one.



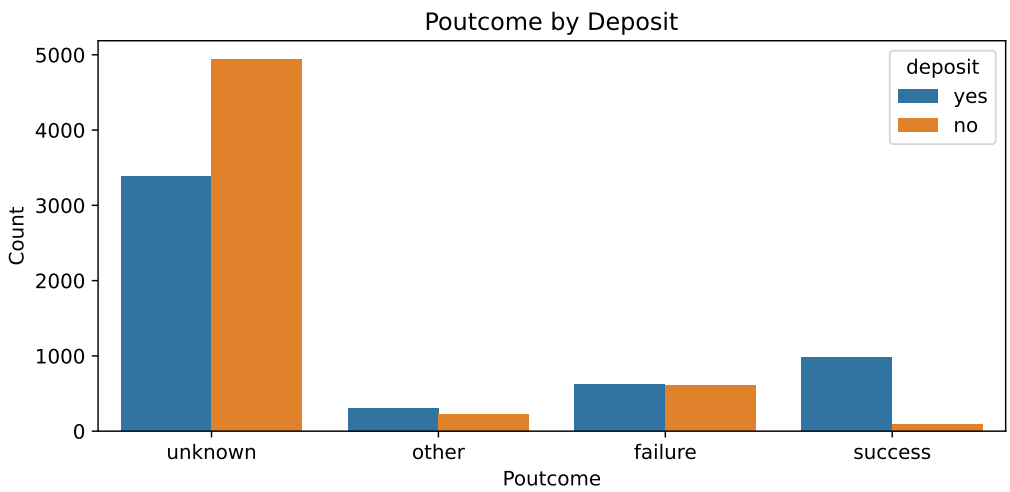

Personal loan status shows a similar pattern: clients without a loan are slightly more likely to subscribe.




The contact communication type reveals that clients contacted by cellular phone subscribe at a higher rate than those contacted by telephone or unknown methods.




Campaign activity varies substantially by month. May has the highest volume but a lower conversion rate, while March, September, October, and December show higher subscription rates despite lower volumes.

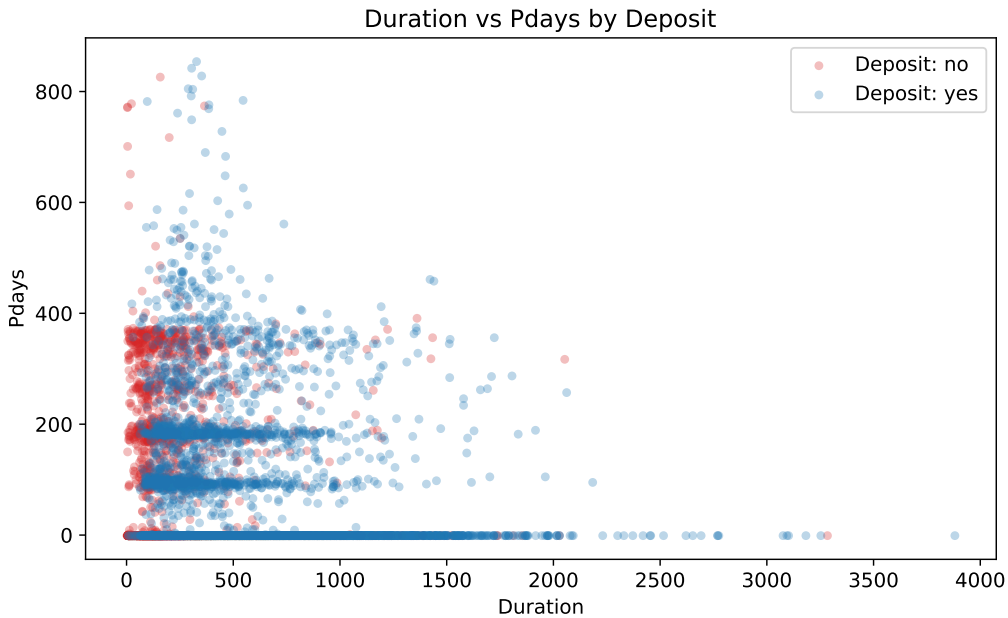


Previous campaign outcome is a strong indicator: clients with a "success" outcome in the prior campaign subscribe at a much higher rate. The majority of clients have an "unknown" previous outcome.

### 3.5 Top-2 correlated numerical features

We examine the relationship between the two numerical features most correlated with the target—`duration` and `pdays`—stratified by deposit class.

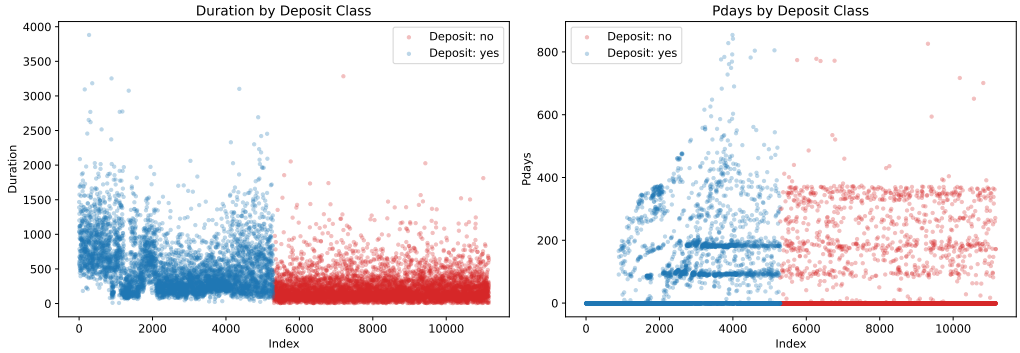
```
colors = {'yes': 'tab:blue', 'no': 'tab:red'}

fig, ax = plt.subplots(figsize=(8, 5))
for label, grp in df.groupby('deposit'):
    ax.scatter(grp[top2[0]], grp[top2[1]], alpha=0.3, s=15,
               color=colors[label], label=f'Deposit: {label}')
ax.set_xlabel(top2[0].capitalize())
ax.set_ylabel(top2[1].capitalize())
ax.set_title(f'{top2[0].capitalize()} vs {top2[1].capitalize()} by Deposit')
ax.legend()
plt.tight_layout()
fig.savefig('figures/fig_05_scatter_top2.pdf', bbox_inches='tight')
plt.show()
```




The scatter plot shows that longer call durations are associated with positive deposit outcomes. Clients who subscribed tend to cluster at higher duration values, while the `pdays` axis reveals that previously contacted clients (positive values) also tend to subscribe more frequently.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for i, feat in enumerate(top2):
    for label, grp in df.groupby('deposit'):
        axes[i].scatter(grp.index, grp[feat], alpha=0.3, s=10,
                        color=colors[label], label=f'Deposit: {label}')
    axes[i].set_xlabel('Index')
    axes[i].set_ylabel(feat.capitalize())
    axes[i].set_title(f'{feat.capitalize()} by Deposit Class')
    axes[i].legend()
```

```
plt.tight_layout()
fig.savefig('figures/fig_06_scatter_individual_top2.pdf', bbox_inches='tight')
plt.show()
```




The individual scatter plots confirm that higher `duration` values and non-negative `pdays` values are more prevalent among clients who subscribed to a term deposit.

# 4 Feature Engineering and Data Splitting

Based on the EDA findings, we apply one-hot encoding to the categorical features, encode the target variable as binary, and split the data into training and test sets with stratification.

## 4.1 One-hot encoding and target encoding

We one-hot encode the nine categorical columns with `drop_first=True` to avoid multicollinearity. The target variable is encoded as 1 for "yes" and 0 for "no."

```
# One-hot encode categorical features
df_encoded = pd.get_dummies(df, columns=categorical_cols,
                            drop_first=True, dtype=int)

# Encode target: yes=1, no=0
df_encoded['deposit'] = (df_encoded['deposit'] == 'yes').astype(int)

# Feature / target split
X = df_encoded.drop('deposit', axis=1).astype(float)
y = df_encoded['deposit']

print(f"Features ({len(X.columns)}): {list(X.columns)}")
```

After one-hot encoding, the feature matrix $\mathbf{X}$ has shape $(11162, 42)$ with 42 columns: 7 original numerical features plus 35 binary indicators from the 9 categorical variables. The target vector $\mathbf{y}$ contains 11,162 binary labels.

## 4.2 Stratified train/test split

We partition the data into training and test sets with stratification on the target variable to preserve the class distribution in both splits.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

```

```python
print(f"Training set: {X_train.shape[0]} samples")
print(f"Test set:     {X_test.shape[0]} samples")
print(f"\nTraining class distribution:\n{y_train.value_counts()}")
print(f"\nTest class distribution:\n{y_test.value_counts()}")
```

```
Training set: 8929 samples
Test set:     2233 samples

Training class distribution:
deposit
0    4698
1    4231

Test class distribution:
deposit
0    1175
1    1058
```

The stratified split ensures that both the training set (52.6% no, 47.4% yes) and the test set preserve the original class ratio. The training set contains 8,929 samples and the test set contains 2,233 samples.

### 4.3 $F_\beta$ score

We use the $F_\beta$ score as the primary evaluation metric. The $F_\beta$ score is defined as

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{precision} \cdot \text{recall}}{\beta^2 \cdot \text{precision} + \text{recall}}.$$

With $\beta = 1$, this reduces to the standard $F_1$ score, which weights precision and recall equally. This is appropriate because the dataset is roughly balanced and the application is not safety-critical.

```python
BETA = 1
fbeta_scorer = make_scorer(fbeta_score, beta=BETA, average='weighted')
print(f"BETA = {BETA}")
print(f"Scorer: F_{BETA} (weighted)")
```

### 4.4 Feature scaling

We standardise all features to zero mean and unit variance so that distance-based and regularised models treat each feature on an equal footing.

```python
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
```

We fit a `StandardScaler` on the training set only, then transform both the training and test sets. Fitting on the training set only prevents information leakage from the test set into the preprocessing step.

## 5 Model Training

We train seven classification models, each with hyperparameter tuning via `GridSearchCV` using five-fold cross-validation and the weighted $F_1$ score as the scoring metric. All models receive the scaled features.

We define a helper function `tune_model` that centralises the grid-search workflow.

```python
best_models = {}
cv_results = []

def tune_model(name, estimator, param_grid):
    """Run GridSearchCV on the training set and store the best estimator."""
    gs = GridSearchCV(estimator, param_grid, cv=5,
                      scoring=fbeta_scorer, n_jobs=-1)
    gs.fit(X_train_sc, y_train)
    best_models[name] = gs.best_estimator_
    cv_results.append({
        'Model': name,
        'Best CV F_beta': gs.best_score_,
        'Best Params': gs.best_params_,
    })
    print(f"{name}:  CV F_{BETA} = {gs.best_score_:.4f},  "
          f"{gs.best_params_}")
```

The function accepts a model name, an unfitted estimator, and a parameter grid. It fits a `GridSearchCV` object on the scaled training data, stores the `best_estimator_` in the dictionary `best_models`, and appends the `best_score_` and `best_params_` to the list `cv_results`.

## 5.1 Logistic regression

We begin with logistic regression, which models the probability of class membership via the logistic function:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{x}^\top \boldsymbol{\beta}) = \frac{1}{1 + e^{-\mathbf{x}^\top \boldsymbol{\beta}}}.$$

The parameters $\boldsymbol{\beta}$ are estimated by maximising the log-likelihood with an optional regularisation penalty. We search over the regularisation strength $C$ and the penalty type.

```python
tune_model('Logistic Regression',
           LogisticRegression(random_state=42, max_iter=1000), {
    'C': [0.01, 0.1, 1.0, 10.0],
    'penalty': ['none', 'l1', 'l2'],
    'solver': ['liblinear', 'saga']
})
```

| Parameter | Values |
|---|---|
| `C` | 0.01, 0.1, 1.0, 10.0 |
| `penalty` | none, l1, l2 |
| `solver` | liblinear, saga |

The best configuration is $C = 0.1$ with $\ell_1$ penalty and the liblinear solver, achieving a cross-validation $F_1$ of 0.8233.

## 5.2 K-nearest neighbours

We train a $k$-nearest neighbours (KNN) classifier, which assigns a test point to the majority class among its $k$ nearest training neighbours. The prediction rule is:

$$\hat{y}(\mathbf{x}) = \arg\max_c \sum_{i \in \mathcal{N}_k(\mathbf{x})} w_i \, \mathbf{1}[y_i = c],$$

where $\mathcal{N}_k(\mathbf{x})$ denotes the set of $k$ nearest neighbours and $w_i$ are weights (uniform or distance-based). We tune $k$, the weighting scheme, and the distance metric.

```
tune_model('KNN', KNeighborsClassifier(), {
    'n_neighbors': [3, 5, 7, 11],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan']
})
```

| Parameter | Values |
|---|---|
| `n_neighbors` | 3, 5, 7, 11 |
| `weights` | uniform, distance |
| `metric` | euclidean, manhattan |

The best KNN uses $k = 11$ with distance weighting and the Manhattan metric, achieving a cross-validation $F_1$ of 0.7722.

### 5.3 Support vector classifier

We train a support vector classifier (SVC) that finds the maximum-margin separating hyperplane in a feature space induced by a kernel function $K$. Given a feature map $\phi$ such that $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$, the primal objective is

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \max\bigl(0,\, 1 - y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b)\bigr),$$

where $C$ controls the trade-off between margin width and training error. The kernel trick allows the optimisation to be solved in the dual without explicitly computing $\phi$. For the RBF kernel, $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma\|\mathbf{x}_i - \mathbf{x}_j\|^2)$, where $\gamma$ controls the width of the Gaussian. We search over $C$, the kernel type, and the kernel coefficient $\gamma$.

```
tune_model('SVC', SVC(random_state=42, probability=True), {
    'C': [0.1, 1.0, 10.0],
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 'auto']
})
```

| Parameter | Values |
|---|---|
| `C` | 0.1, 1.0, 10.0 |
| `kernel` | linear, rbf |
| `gamma` | scale, auto |

The best SVC uses $C = 1.0$ with the RBF kernel and $\gamma =$ auto, achieving a cross-validation $F_1$ of 0.8396.

### 5.4 Decision tree

We train a decision tree classifier $T(\mathbf{x};\, d_{\max},\, n_{\min})$, where $d_{\max}$ is the maximum depth and $n_{\min}$ is the minimum number of samples required to split an internal node. At each node the tree selects the feature $j$ and threshold $t$ that minimise the impurity of the resulting partition:

$$\min_{j,\,t} \left[\frac{n_L}{n}\,\mathrm{Gini}(R_L) + \frac{n_R}{n}\,\mathrm{Gini}(R_R)\right], \quad \mathrm{Gini}(R) = 1 - \sum_c p_c^2.$$

We tune $d_{\max}$, $n_{\min}$, and the splitting criterion.

```python
tune_model('Decision Tree', DecisionTreeClassifier(random_state=42), {
    'max_depth': [3, 5, 7, 10],
    'min_samples_split': [2, 5, 10],
    'criterion': ['gini', 'entropy']
})
```

| Parameter | Values |
|---|---|
| `max_depth` | 3, 5, 7, 10 |
| `min_samples_split` | 2, 5, 10 |
| `criterion` | gini, entropy |

The best tree uses the Gini criterion with `max_depth` = 7 and `min_samples_split` = 5, achieving a cross-validation $F_1$ of 0.8132.

### 5.5 Random forest

We train a random forest, an ensemble of decorrelated decision trees. The ensemble prediction aggregates $B$ independently trained trees, each fitted on a bootstrap sample:

$$\hat{y} = \text{majority vote}\big(T_1(\mathbf{x}),\, T_2(\mathbf{x}),\, \ldots,\, T_B(\mathbf{x})\big),$$

where $T_b$ denotes the $b$-th decision tree. Each tree considers a random subset of features at each split to decorrelate the trees. We tune the number of estimators $B$, $d_{\max}$, and the minimum number of samples per leaf.

```python
tune_model('Random Forest', RandomForestClassifier(random_state=42), {
    'n_estimators': [100, 200],
    'max_depth': [5, 10, None],
    'min_samples_leaf': [1, 4]
})
```

| Parameter | Values |
|---|---|
| `n_estimators` | 100, 200 |
| `max_depth` | 5, 10, None |
| `min_samples_leaf` | 1, 4 |

The best random forest uses 200 trees with no maximum depth restriction and `min_samples_leaf` = 1, achieving a cross-validation $F_1$ of 0.8496.

### 5.6 Gradient boosting

We train a gradient boosting classifier, which builds $M$ trees sequentially to correct the errors of previous trees. At stage $m$ the ensemble is

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta\, h_m(\mathbf{x}), \quad m = 1, \ldots, M,$$

where $h_m$ is a decision tree of maximum depth $d_{\max}$ fitted to the negative gradient of the loss, $\eta$ is the learning rate, and $M$ is the total number of boosting stages. We tune $\eta$, $M$, and $d_{\max}$.

```python
tune_model('Gradient Boosting',
           GradientBoostingClassifier(random_state=42), {
    'n_estimators': [100, 200],
    'learning_rate': [0.05, 0.1],
    'max_depth': [3, 5]
})
```

| Parameter | Values |
|---|---|
| `n_estimators` | 100, 200 |
| `learning_rate` | 0.05, 0.1 |
| `max_depth` | 3, 5 |

The best gradient boosting configuration uses 200 trees with a learning rate of 0.1 and `max_depth` = 5, yielding a cross-validation $F_1$ of 0.8577—the highest among all seven models.

### 5.7 Gaussian naive Bayes

We train a Gaussian naive Bayes classifier, which assumes that the features are conditionally independent given the class label and follow Gaussian distributions:

$$P(\mathbf{x} \mid y = c) = \prod_{j=1}^{p} \frac{1}{\sqrt{2\pi\sigma_{jc}^2}} \exp\left(-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2}\right).$$

The predicted class is obtained via Bayes' theorem: $\hat{y} = \arg\max_c P(y = c) \prod_j P(x_j \mid y = c)$. We tune the variance smoothing parameter $\alpha$, which is added to the estimated variances for numerical stability.

```
tune_model('Gaussian NB', GaussianNB(), {
    'var_smoothing': [1e-9, 1e-8, 1e-7, 1e-6]
})
```

| Parameter | Values |
|---|---|
| `var_smoothing` | $10^{-9}$, $10^{-8}$, $10^{-7}$, $10^{-6}$ |

The best variance smoothing is $10^{-9}$ (the default), achieving a cross-validation $F_1$ of 0.6982. The low score reflects the strong conditional independence assumption, which is violated by the correlated features in this dataset.

## 6 Model Evaluation

We evaluate each tuned model on the held-out test set using three metrics: **accuracy**, the weighted $F_1$ **score**, and the **ROC-AUC**. For each model, we compute predictions on the scaled test features `X_test_sc`.

```
model_names = list(best_models.keys())
test_preds = {}
test_probs = {}

for name in model_names:
    model = best_models[name]
    y_pred = model.predict(X_test_sc)
    test_preds[name] = y_pred

    if hasattr(model, 'predict_proba'):
        test_probs[name] = model.predict_proba(X_test_sc)[:, 1]
    else:
        test_probs[name] = model.decision_function(X_test_sc)

    acc = accuracy_score(y_test, y_pred)
    fb = fbeta_score(y_test, y_pred, beta=BETA, average='weighted')
```

```
    ra = roc_auc_score(y_test, test_probs[name])
    print(f"{name}")
    print(f"  Accuracy:  {acc:.4f}")
    print(f"  F_{BETA}:      {fb:.4f}")
    print(f"  ROC-AUC:   {ra:.4f}")
    print()
```

## 6.1 Per-model test results

We now summarise the test-set performance of each model individually, highlighting the effect of model complexity and inductive bias on generalization to unseen data.

- **Logistic regression** achieves a test accuracy of 0.8240, $F_1$ of 0.8238, and ROC-AUC of 0.9082. The $\ell_1$ penalty with $C = 0.1$ provides moderate regularisation, yielding solid baseline performance.
- **KNN** achieves a test accuracy of 0.7815, $F_1$ of 0.7795, and ROC-AUC of 0.8696. The relatively large number of features (42) dilutes the effectiveness of distance-based methods.
- **SVC** achieves a test accuracy of 0.8491, $F_1$ of 0.8492, and ROC-AUC of 0.9187. The RBF kernel captures non-linear decision boundaries effectively.
- **Decision tree** achieves a test accuracy of 0.8142, $F_1$ of 0.8141, and ROC-AUC of 0.8857. The limited depth of 7 controls overfitting but also limits capacity.
- **Random forest** achieves a test accuracy of 0.8585, $F_1$ of 0.8586, and ROC-AUC of 0.9204. The ensemble of 200 unrestricted trees reduces variance compared to a single decision tree.
- **Gradient boosting** achieves the highest test accuracy of 0.8661, $F_1$ of 0.8662, and ROC-AUC of 0.9302. The sequential error-correction mechanism proves most effective on this dataset.
- **Gaussian NB** achieves a test accuracy of 0.7179, $F_1$ of 0.7079, and ROC-AUC of 0.8101. The conditional independence assumption limits its ability to model feature interactions.

## 6.2 Summary of test results

Table 1 consolidates the test-set performance of all seven models.

Table 1: Test-set performance of all seven models.

| Model | Accuracy | $F_1$ | ROC-AUC |
|---|---|---|---|
| Gradient Boosting | 0.8661 | 0.8662 | 0.9302 |
| Random Forest | 0.8585 | 0.8586 | 0.9204 |
| SVC | 0.8491 | 0.8492 | 0.9187 |
| Logistic Regression | 0.8240 | 0.8238 | 0.9082 |
| Decision Tree | 0.8142 | 0.8141 | 0.8857 |
| KNN | 0.7815 | 0.7795 | 0.8696 |
| Gaussian NB | 0.7179 | 0.7079 | 0.8101 |

Table 2: Full model comparison: cross-validation $F_1$, test accuracy, test $F_1$, and test ROC-AUC.

| Model | CV $F_1$ | Accuracy | Test $F_1$ | ROC-AUC |
|---|---|---|---|---|
| Gradient Boosting | 0.8577 | 0.8661 | 0.8662 | 0.9302 |
| Random Forest | 0.8496 | 0.8585 | 0.8586 | 0.9204 |
| SVC | 0.8396 | 0.8491 | 0.8492 | 0.9187 |
| Logistic Regression | 0.8233 | 0.8240 | 0.8238 | 0.9082 |
| Decision Tree | 0.8132 | 0.8142 | 0.8141 | 0.8857 |
| KNN | 0.7722 | 0.7815 | 0.7795 | 0.8696 |
| Gaussian NB | 0.6982 | 0.7179 | 0.7079 | 0.8101 |

# 7 Model Comparison

We consolidate cross-validation and test-set results into a single comparison table and visualise the differences.

```
rows = []
for rec in cv_results:
    name = rec['Model']
    yp = test_preds[name]
    rows.append({
        'Model':         name,
        'CV F_beta':     rec['Best CV F_beta'],
        'Test Accuracy': accuracy_score(y_test, yp),
        'Test F_beta':   fbeta_score(y_test, yp, beta=BETA,
                                     average='weighted'),
        'Test ROC-AUC':  roc_auc_score(y_test, test_probs[name]),
    })

comparison_df = pd.DataFrame(rows).sort_values('Test F_beta',
                                                ascending=False)
```

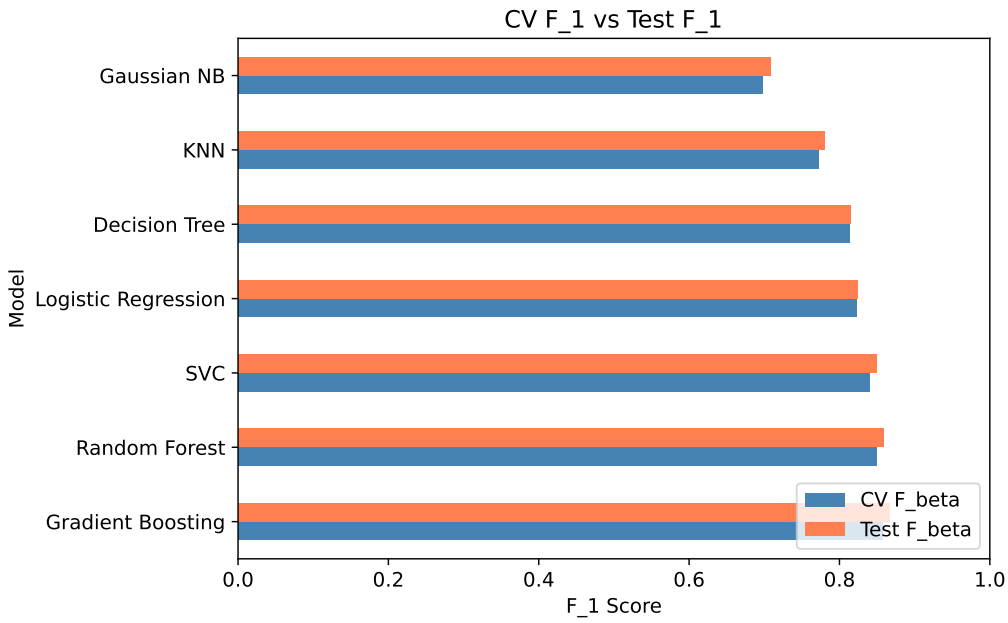
The DataFrame `comparison_df` contains five columns—`Model`, `CV` $F_1$, `Test Accuracy`, `Test` $F_1$, and `Test ROC-AUC`—sorted by descending test $F_1$. Table 2 presents the full comparison.

Gradient boosting ranks first across all three test metrics. Random forest and SVC follow closely. The three distance-based or assumption-heavy models (KNN, Gaussian NB) lag behind, while logistic regression and decision tree occupy the middle ground.

## 7.1 CV $F_1$ vs test $F_1$

We visualise the cross-validation and test $F_1$ scores side by side to assess whether any model overfits or underfits relative to its training performance.

```
fig, ax = plt.subplots(figsize=(8, 5))
plot_df = comparison_df.set_index('Model')[['CV F_beta', 'Test F_beta']]
plot_df.plot(kind='barh', ax=ax, color=['steelblue', 'coral'])
ax.set_xlabel(f'F_{BETA} Score')
ax.set_title(f'CV F_{BETA} vs Test F_{BETA}')
ax.set_xlim(0, 1)
ax.legend(loc='lower right')
plt.tight_layout()
fig.savefig('figures/fig_07a_model_fbeta_comparison.pdf',
            bbox_inches='tight')
plt.show()
```

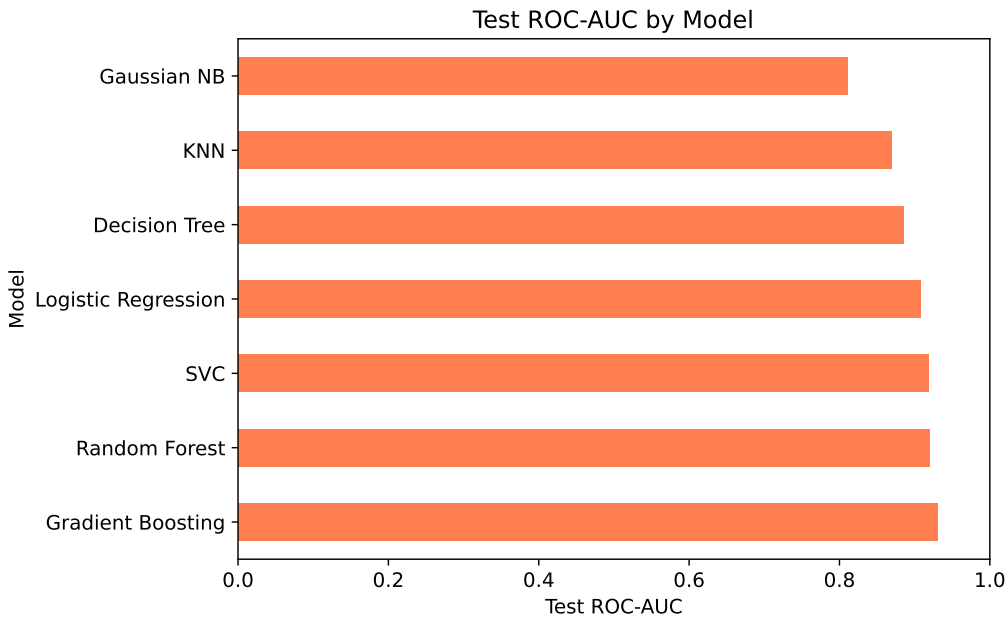



Every model achieves a test $F_1$ close to or slightly above its cross-validation $F_1$, suggesting that the models generalise well and the particular test split is representative of the overall data distribution.

## 7.2 Test ROC-AUC comparison

We compare the test ROC-AUC across all models to identify which model best discriminates between the two classes.

```
fig, ax = plt.subplots(figsize=(8, 5))
comparison_df.set_index('Model')['Test ROC-AUC'].plot(
    kind='barh', ax=ax, color='coral'
)
ax.set_xlabel('Test ROC-AUC')
ax.set_title('Test ROC-AUC by Model')
ax.set_xlim(0, 1)
plt.tight_layout()
fig.savefig('figures/fig_07b_model_roc_auc_comparison.pdf',
            bbox_inches='tight')
plt.show()
```

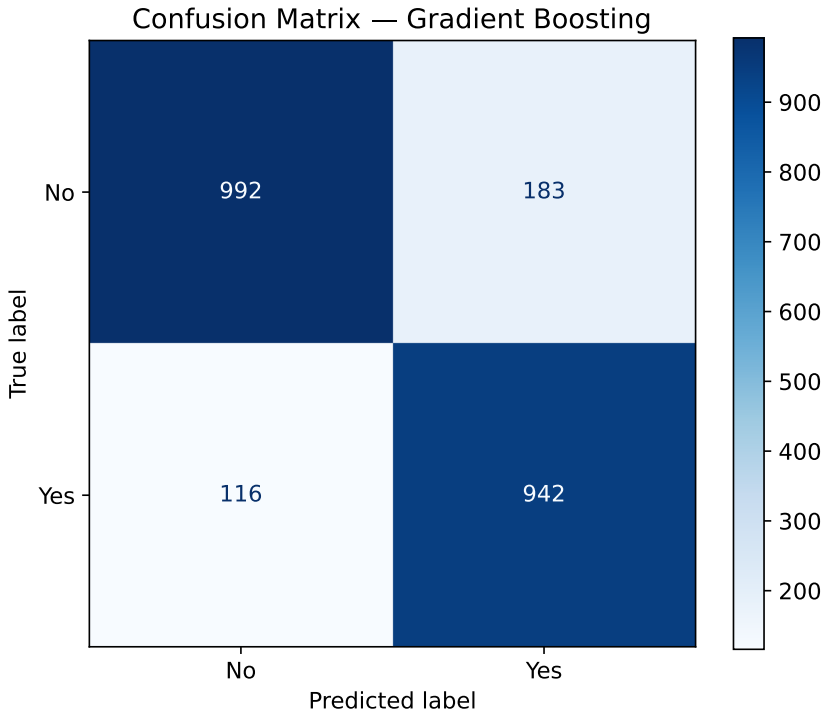



Gradient boosting achieves the highest ROC-AUC of 0.9302, indicating excellent discrimination between deposit subscribers and non-subscribers across all classification thresholds.

### 7.3 Confusion matrix

We plot the confusion matrix of the best model to examine the types of classification errors.

```
fig, ax = plt.subplots(figsize=(6, 5))
cm = confusion_matrix(y_test, test_preds[best_name])
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=['No', 'Yes'])
disp.plot(ax=ax, cmap='Blues')
ax.set_title(f'Confusion Matrix | {best_name}')
plt.tight_layout()
fig.savefig('figures/fig_08_confusion_matrix.pdf', bbox_inches='tight')
plt.show()
```

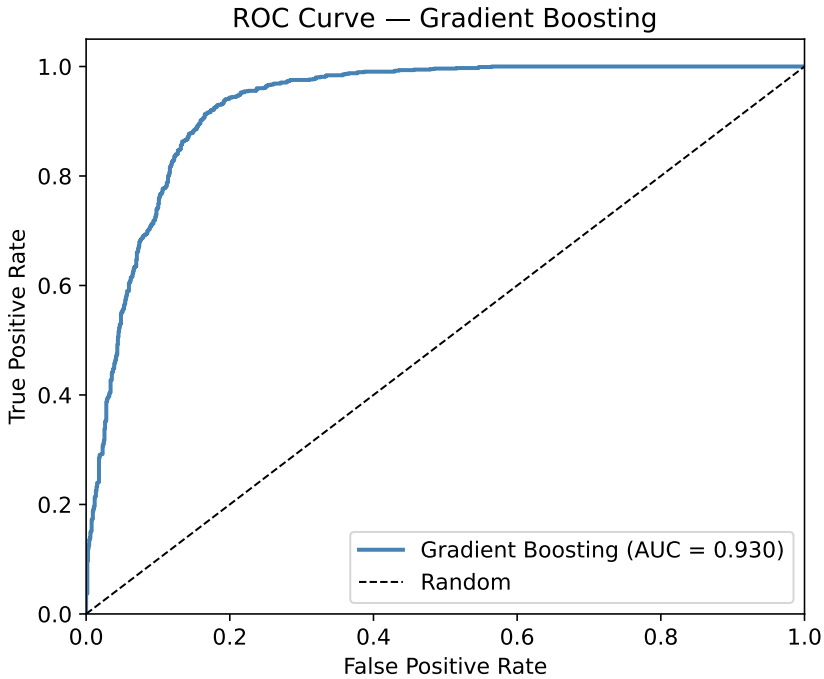



The confusion matrix shows that the gradient boosting classifier correctly identifies the majority of both classes. The false positive and false negative rates are roughly balanced, consistent with the similar accuracy and $F_1$ values observed.

### 7.4 ROC curve

We plot the receiver operating characteristic (ROC) curve of the best model.

```
fpr, tpr, _ = roc_curve(y_test, test_probs[best_name])
roc_auc_val = auc(fpr, tpr)

fig, ax = plt.subplots(figsize=(6, 5))
ax.plot(fpr, tpr, color='steelblue', lw=2,
        label=f'{best_name} (AUC = {roc_auc_val:.3f})')
ax.plot([0, 1], [0, 1], 'k--', lw=1, label='Random')
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.set_title(f'ROC Curve | {best_name}')
ax.legend(loc='lower right')
ax.set_xlim([0, 1])
ax.set_ylim([0, 1.05])
plt.tight_layout()
fig.savefig('figures/fig_09_roc_curve.pdf', bbox_inches='tight')
plt.show()
```




The ROC curve hugs the top-left corner, achieving an AUC of 0.930. The model maintains high true positive rates even at low false positive rates, confirming its strong discriminative ability.

### 7.5 Best model selection

We select the best model based on the highest test $F_1$ score.

```
best_row = comparison_df.iloc[0]
best_name = best_row['Model']
best_model = best_models[best_name]

print(f"Best model: {best_name}")
print(f"  CV F_{BETA}:       {best_row['CV F_beta']:.4f}")
print(f"  Test Accuracy: {best_row['Test Accuracy']:.4f}")
print(f"  Test F_{BETA}:     {best_row['Test F_beta']:.4f}")
print(f"  Test ROC-AUC:  {best_row['Test ROC-AUC']:.4f}")
```

We select **gradient boosting** as the best model based on the highest test $F_1$ of 0.8662. Its test accuracy is 0.8661 and its ROC-AUC is 0.9302. The model correctly classifies approximately 87% of clients on unseen data.

## 8 Model Deployment

We serialise the best model, the scaler, and the feature list into a single pickle file for future inference.

### 8.1 Saving the artifact

We bundle the trained model, the fitted scaler, the ordered feature list, and the $\beta$ parameter into a single dictionary and serialise it using Python's `pickle` module.

```python
artifact = {
    'model': best_model,
    'model_name': best_name,
    'scaler': scaler,
    'features': list(X.columns),
    'beta': BETA,
}

with open('bank_model.pkl', 'wb') as f:
    pickle.dump(artifact, f)

print(f"Saved '{best_name}' to bank_model.pkl")
```

The dictionary `artifact` contains five keys:

- `model` — the fitted `GradientBoostingClassifier` instance with 200 trees, learning rate 0.1, and `max_depth` = 5.
- `model_name` — the string `"Gradient Boosting"`.
- `scaler` — the fitted `StandardScaler` that learned the training-set mean and standard deviation for each feature.
- `features` — the ordered list of 42 feature names.
- `beta` — the value 1 used for the $F_\beta$ scorer.

Bundling these together ensures that any future user can reproduce the exact preprocessing and prediction pipeline without access to the training data.

## 8.2 Loading and inspecting the artifact

We reload the serialised artifact from disk to verify that the model, scaler, and feature list were persisted correctly.

```python
with open('bank_model.pkl', 'rb') as f:
    artifact = pickle.load(f)

print(f"Keys:       {list(artifact.keys())}")
print(f"Model name: {artifact['model_name']}")
print(f"Model type: {type(artifact['model']).__name__}")
print(f"Beta:       {artifact['beta']}")
```

```
Keys:       ['model', 'model_name', 'scaler', 'features', 'beta']
Model name: Gradient Boosting
Model type: GradientBoostingClassifier
Beta:       1
```

```python
print("Features:", artifact['features'])
print(f"\nScaler:   {type(artifact['scaler']).__name__}")
print(f"  mean:   {np.round(artifact['scaler'].mean_, 2)}")
print(f"  scale:  {np.round(artifact['scaler'].scale_, 2)}")
```

The scaler's learned means and standard deviations confirm reasonable training-set statistics. The 42 features include the 7 original numerical columns plus 35 one-hot-encoded binary indicators.

### 8.3 Prediction on a new observation

We demonstrate the end-to-end inference pipeline by constructing a single synthetic observation, scaling it with the stored scaler, and generating a predicted class.

```python
# Construct a single unseen observation
# A 35-year-old married technician with secondary education,
# no default, positive balance, housing loan, no personal loan,
# contacted by cellular in June.
new_obs = pd.DataFrame([{col: 0 for col in artifact['features']}])
new_obs['age'] = 35
new_obs['balance'] = 1500
new_obs['day'] = 15
new_obs['duration'] = 300
new_obs['campaign'] = 2
new_obs['pdays'] = -1
new_obs['previous'] = 0
# Set appropriate one-hot encoded columns
for col in new_obs.columns:
    if col == 'job_technician':
        new_obs[col] = 1
    elif col == 'marital_married':
        new_obs[col] = 1
    elif col == 'education_secondary':
        new_obs[col] = 1
    elif col == 'housing_yes':
        new_obs[col] = 1
    elif col == 'month_jun':
        new_obs[col] = 1

print("Input (raw):")
print(new_obs.to_string(index=False))

new_obs_sc = artifact['scaler'].transform(new_obs)
predicted_class = artifact['model'].predict(new_obs_sc)[0]
predicted_label = 'yes' if predicted_class == 1 else 'no'

if hasattr(artifact['model'], 'predict_proba'):
    prob = artifact['model'].predict_proba(new_obs_sc)[0]
    print(f"\nPredicted class: {predicted_label}")
    print(f"Probability (no):  {prob[0]:.4f}")
    print(f"Probability (yes): {prob[1]:.4f}")
else:
    print(f"\nPredicted class: {predicted_label}")
```

```
Predicted class: yes
Probability (no):  0.1757
Probability (yes): 0.8243
```

The synthetic observation describes a 35-year-old married technician with secondary education, an account balance of 1,500 euros, a housing loan, no personal loan, and no credit default. The client was contacted by cellular phone in June for 300 seconds during the second contact of the campaign. The gradient boosting classifier predicts that this client will subscribe to a term deposit with a probability of 0.8243.

# 9 Conclusion

We have completed a data science workflow for predicting whether bank clients will subscribe to a term deposit based on demographic, financial, and campaign-related features.

**Data cleaning.** The dataset contained 11,162 rows with 17 columns. No missing values or duplicate rows were found, so the dataset was used as-is.

**EDA findings.** The target variable is roughly balanced (no = 5,873, yes = 5,289; ratio 0.90). Call duration emerged as the strongest predictor ($|r| = 0.45$), followed by `pdays` (0.15) and `previous` (0.14). Categorical features such as previous campaign outcome, contact method, and job type also show meaningful variation in deposit subscription rates.

**Model comparison.** We trained and tuned seven classification models via five-fold cross-validation and evaluated each on a held-out test set. Gradient boosting achieved the highest test $F_1$ of 0.8662 and ROC-AUC of 0.9302, followed closely by random forest ($F_1 = 0.8586$, ROC-AUC = 0.9204). SVC and logistic regression performed well, while KNN and Gaussian naive Bayes lagged behind due to the high-dimensional one-hot-encoded feature space and violated independence assumptions, respectively.

**Practical implications.** The analysis reveals which client profiles are most likely to subscribe to a term deposit, enabling targeted marketing campaigns. The strong influence of call duration suggests that longer, more engaged conversations lead to higher conversion rates. Previous campaign success is also a strong indicator, highlighting the value of maintaining client relationship histories. The exported pickle artifact enables real-time prediction for new clients.

**Limitations.** The dataset does not include economic indicators (e.g., interest rates, employment rate) that may influence deposit subscription decisions. The `duration` feature is known only after a call ends and therefore cannot be used as a true predictor in a production setting—it serves here as a proxy for client engagement. Additionally, the one-hot encoding of categorical features with many levels (e.g., `job` with 12 categories, `month` with 12 months) inflates the feature space to 42 dimensions, which may affect the performance of distance-based methods.

**Future work.** Several directions could improve performance:

(i) Incorporating macroeconomic features such as the consumer confidence index and inter-bank lending rates.

(ii) Exploring target encoding or ordinal encoding as alternatives to one-hot encoding to reduce dimensionality.

(iii) Testing additional ensemble methods such as XGBoost or LightGBM with more extensive hyperparameter grids.

(iv) Investigating the causal effect of call duration by building models that exclude it, to assess predictive power from features available before the call.

# References

[1] C. R. Harris et al. "Array programming with NumPy". In: *Nature* 585.7825 (Sept. 2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2.

[2] J. D. Hunter. "Matplotlib: A 2D graphics environment". In: *Computing in Science & Engineering* 9.3 (2007), pp. 90–95.

[3] W. McKinney. "Data structures for statistical computing in Python". In: *Proceedings of the 9th Python in Science Conference*. Vol. 445. 2010, pp. 51–56.

[4] S. Moro, P. Cortez, and P. Rita. "A data-driven approach to predict the success of bank telemarketing". In: *Decision Support Systems* 62 (2014), pp. 22–31. DOI: 10.1016/j.dss.2014.03.001.

[5] S. Moro, P. Rita, and P. Cortez. *Bank Marketing*. UCI Machine Learning Repository. 2014. DOI: 10.24432/C5K306.

[6] F. Pedregosa et al. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.

[7] Python Software Foundation. *pickle — Python object serialization*. https://docs.python.org/3/library/pickle.html. Accessed: 2026-03-04. 2026.

[8] M. L. Waskom. "seaborn: statistical data visualization". In: *Journal of Open Source Software* 6.60 (2021), p. 3021. DOI: 10.21105/joss.03021.